
CS 421 --- Higher Order Functions Activity

Name	Netid	
Name	Netid	
Name	Netid	

Learning Objectives

Mapping, folding, and zipping allow us to abstract away common list computations. Knowing how to use them will make you more productive as a programmer.

1. Reduce code size by using `map`, `foldr`, and `zipWith`.
2. Use type signatures to implement `curry`, `uncurry`, and `flip`.

Mapping (5 minutes)

Consider the following code and sample run, stolen from *Haskell Programming From First Principles*, Chapter 9.

```
0 Prelude> map (+1) [1, 2, 3, 4]
1 [2,3,4,5]
2 Prelude> map (1-) [1, 2, 3, 4]
3 [0,-1,-2,-3]
4 Prelude> map (take 2) [[1, 4, 9], [2, 3, 5], [11, 12, 13]]
5 [[1,4],[2,3],[11,12]]
6 Prelude> zipWith (+) [1, 2, 3] [10, 11, 12]
7 [11,13,15]
8 Prelude> zipWith max [10, 5, 34, 9] [6, 8, 12, 7]
9 [10,8,34,9]
```

Problem 1) What does the `(+1)` code mean?

Problem 2) In the second example, why are we using `(1-)` and not `(-1)`?

Problem 3) How would you describe the difference between `map` and `zipWith`?

Reducing

Haskell has some functions `foldr` and `foldl` that behave like the `reduce` function found in other languages such as Python and JavaScript. Perhaps you have used it!

Consider the following sample run.

```
0 Prelude> :set +m
1 Prelude> { showPair x "" = x
2 Prelude| ; showPair x y = "(" ++ x ++ ", " ++ y ++ ")"
3 Prelude| }
4 Prelude> showPair "10" ""
5 "10"
6 Prelude> showPair "10" "20"
7 "(10,20)"
```

Problem 4) What do you think the `:set +m`, curly braces, and semicolons are about? What happens if we don't use them?

Problem 5) What does `showPair` do?

Now consider this run:

```
0 Prelude> sumList1 xx = foldr (+) 0 xx
1 Prelude> sumList1 [3,4,5,6]
2 18
3 Prelude> sumList2 xx = foldl (+) 0 xx
4 Prelude> sumList2 [3,4,5,6]
5 18
6 Prelude> pairList1 xx = foldr showPair "" xx
7 Prelude> pairList2 xx = foldl showPair "" xx
8 Prelude> pairList1 ["3","4","5","6"]
9 "(3,(4,(5,6)))"
10 Prelude> pairList2 ["3","4","5","6"]
11 "((((3),4),5),6)"
```

Problem 6) Is there an observable difference between `sumList1` and `sumList2`?

Problem 7) Is there an observable difference between `pairList1` and `pairList2`?

Problem 8) Write a function `prodList` using the same technique you see here.

Problem 9) Do you think either `foldr` or `foldl` is tail recursive? Why or why not?

List Comprehensions

List comprehensions are similar to higher order functions, and can allow you to write very compact code.

```
0 Prelude> [x+1 | x <- [1..10]]
1 [2,3,4,5,6,7,8,9,10,11]
2 Prelude> [x+1 | x <- [1..10], x>5]
3 [7,8,9,10,11]
4 Prelude> stuff = [8,6,7,5,3,0,9]
5 Prelude> [ x+1 | x <- stuff ]
6 [9,7,8,6,4,1,10]
7 Prelude> [ x+1 | x <- stuff, x > 5]
8 [9,7,8,10]
9 Prelude> [ x+1 | x <- stuff, x > 5, even x]
10 [9,7]
11 Prelude> [ x + y | x <- stuff, y <- [10,20]]
12 [18,28,16,26,17,27,15,25,13,23,10,20,19,29]
```

Problem 10) What is the purpose of the `x <- stuff` expression?

Problem 11) What is the purpose of `x > 5`, and `even x`?

Problem 12) How do you describe the order in which `x` and `y` are created in the last example?

Problem 13) What does the following code do?

```
0 guess [] = []
1 guess (x:xs) = guess [y | y <- xs, y < x]
2               ++ [x] ++
3               guess [y | y <- xs, y >= x]
```

Currying

Consider these two functions:

```
0 Prelude> uplus (a,b) = a + b
1 Prelude> cplus a b = a + b
```

Problem 14) What is the difference between `cplus` and `uplus`? What would it look like to use them?

The function `cplus`, which is written in idiomatic Haskell, is said to be *curried*. This makes it taste better.

Problem 15) Write a function `curry :: ((a,b) -> c) -> a -> b -> c` that takes a non-curried function and returns an equivalent curried version.

```
0 Prelude> plus (a,b) = a + b
1 Prelude> :t plus
2 Num a => (a,a) -> a
3 Prelude> cplus = curry plus
4 Prelude> cplus 10 20
5 30
```

Problem 16) Write a function `flip :: (a -> b -> c) -> (b -> a -> c)` that takes a function that takes two arguments and returns an equivalent function where the arguments have been reversed.

```
0 Prelude> sub a b = a - b
1 Prelude> flip sub 10 2
2 -8
```

Problem 17) Consider the types of `flip` and `curry`. Can you write another function that has either of those types? Why or why not?